Build a Lakehouse for E-commerce Transactions using Delta Lake on AWS

Lab Summary

This lab demonstrates how to build a **modern Lakehouse architecture** on AWS using Delta Lake. You will ingest raw transactional data into Amazon S3, process it using Spark, create Delta tables, catalog them using AWS Glue Crawlers, query using Athena, and understand Redshift integration.

You will also implement **upsert (merge)** operations to update existing data and validate ACID capabilities of Delta Lake.

Prerequisites to Complete Lab

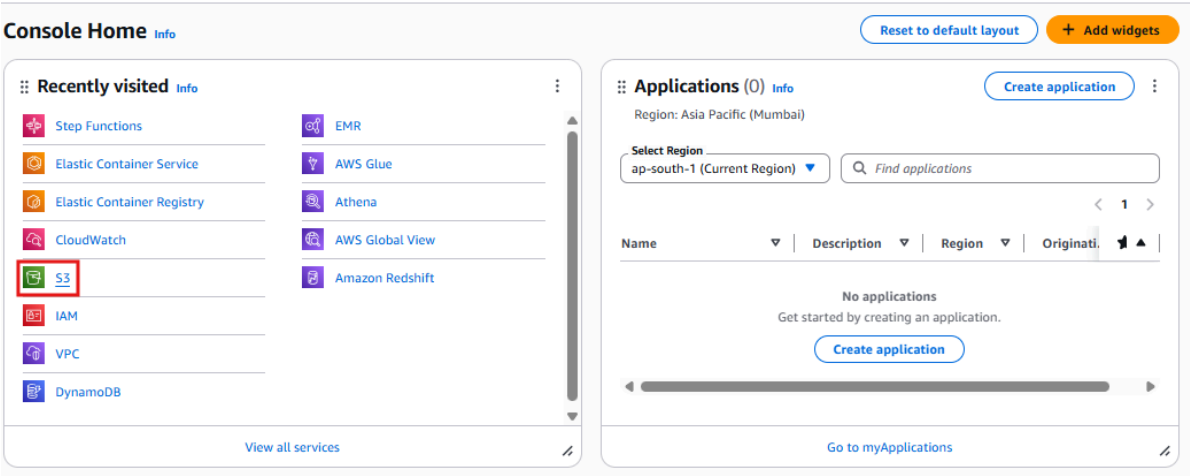
- AWS account with console access
- S3 bucket created
- IAM role with:
  - S3 full access
  - CloudWatch logs access
- Docker installed locally
- Input datasets:
  - products CSV
  - orders CSV
  - order_items CSV
- Spark script
- Dockerfile
- Delta Lake jar file

- Basic knowledge of AWS services

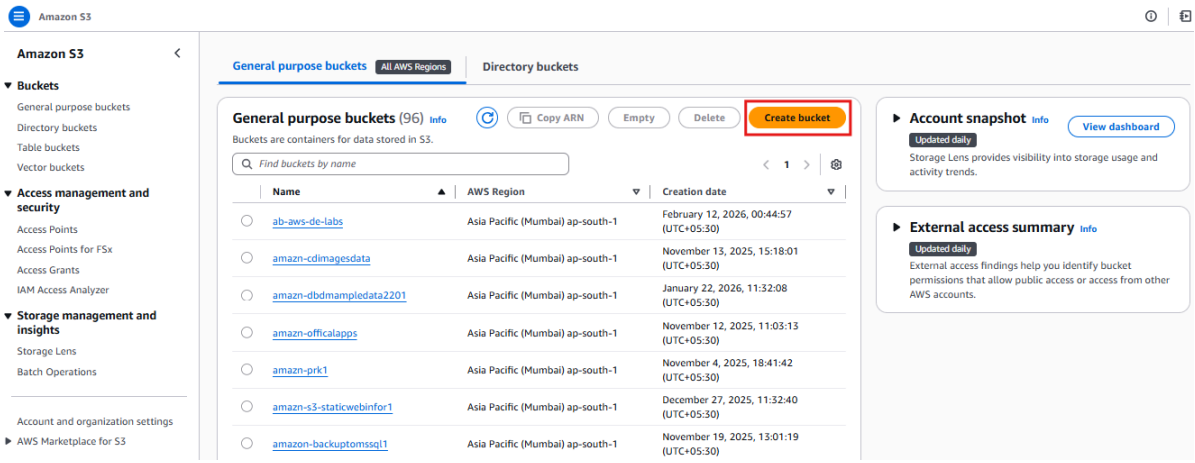
Steps to Complete Lab

Step 1 — Upload Raw Data to S3

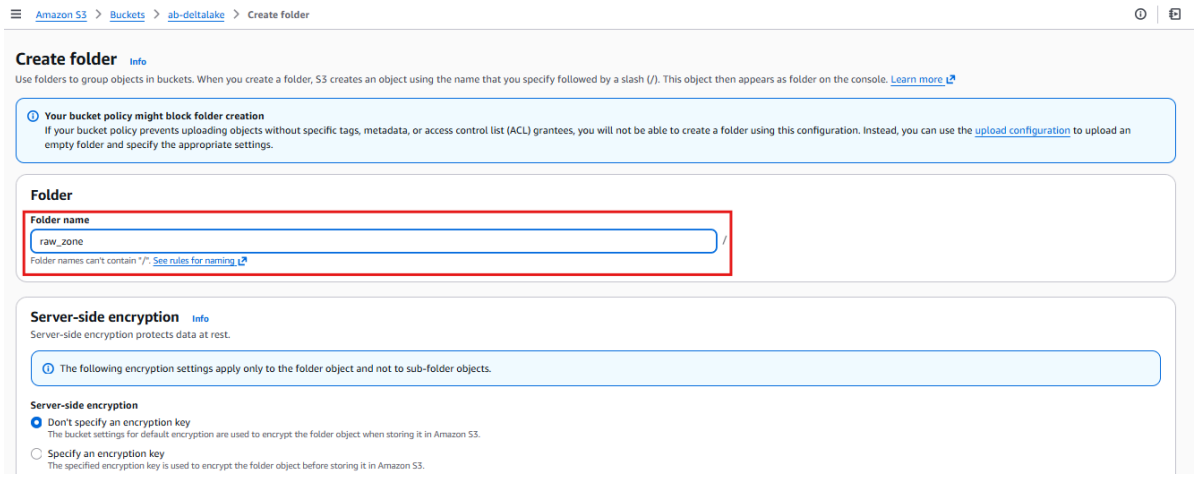
1. Open AWS Console. —→ Go to S3



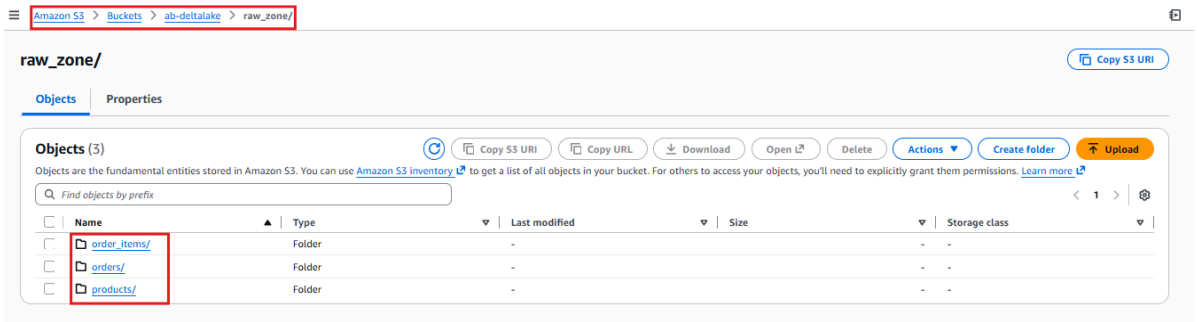
2. Create your bucket —→ Open your bucket



3. Create folder → **raw_zone**.



4. Inside raw_zone create folders:
   - orders
   - order_items
   - products



5. Upload respective CSV files into each folder.

Step 4 — Review Spark Script

1. Open spark script.
2. Verify imports including DeltaTable.
3. Confirm S3 bucket name is correct.
4. Check input path = raw_zone.
5. Check output path = lakehouse_dw.
6. Confirm merge logic using primary keys.

7. Confirm partitioning using order_date.
8. Verify archive logic moves processed files.

Step 5 — Build Docker Image

1. Open terminal.
2. Navigate to project directory.
3. Ensure these files exist:
   - Dockerfile
   - spark script
   - delta jar
4. Run command:

   docker build -t spark-delta-job .
5. Wait for image to build successfully.

Step 6 — Run Spark Job Locally

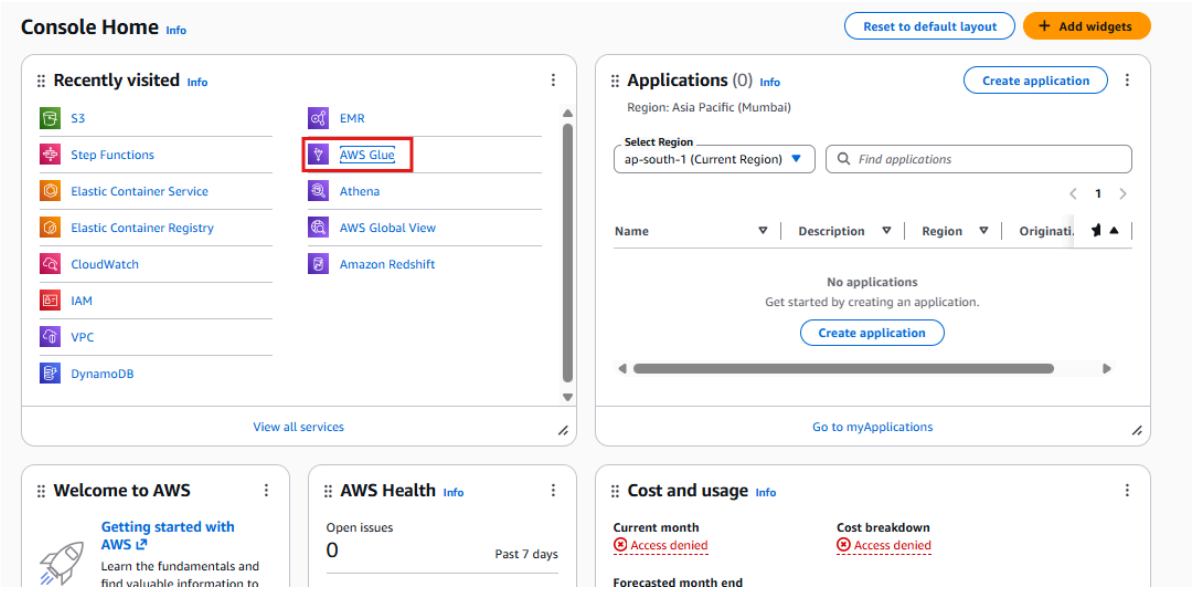
1. Execute docker run command mounting jar file.
2. Run spark script for orders dataset.
3. Wait for execution to complete.
4. Confirm message → files moved to archive successfully.

Step 7 — Verify Output in S3

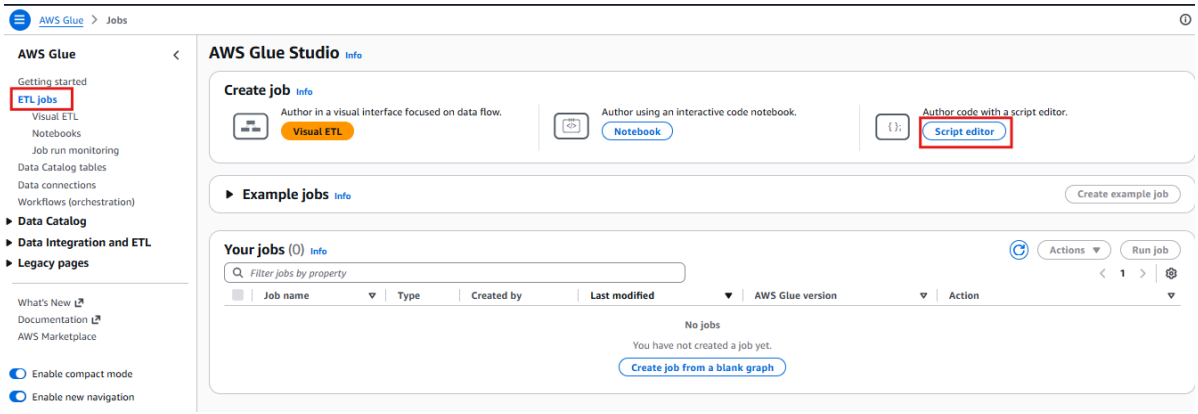
1. Open S3 bucket.
2. Verify folders created:
   - lakehouse_dw
   - archive
3. Inside lakehouse_dw confirm Delta table created.
4. Check partition folder by order_date.

Step 8 — Create AWS Glue ETL Job

1. Go to AWS console —→ AWS Glue console

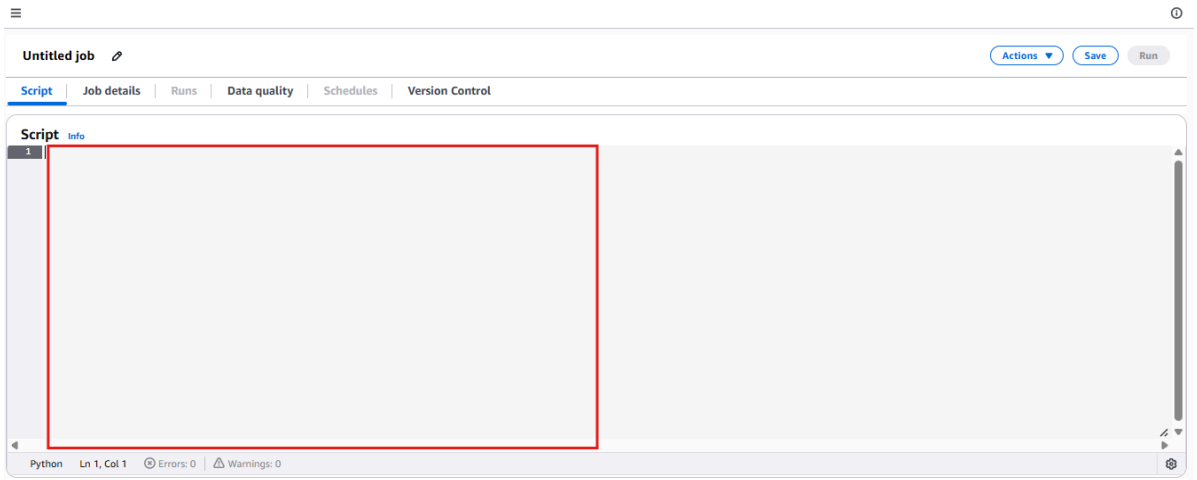


2. Select ETL Jobs → Script editor.



3. Click on Create script

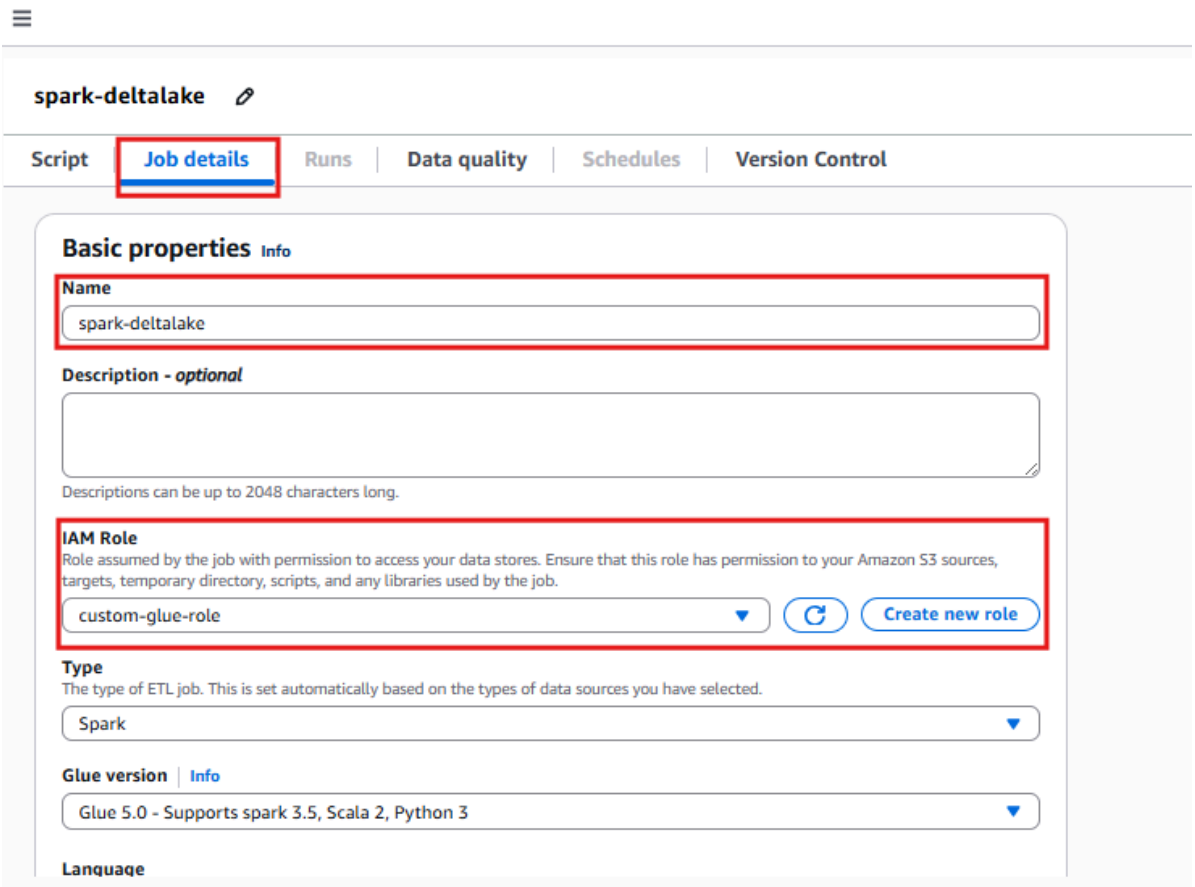
4. Paste spark script.



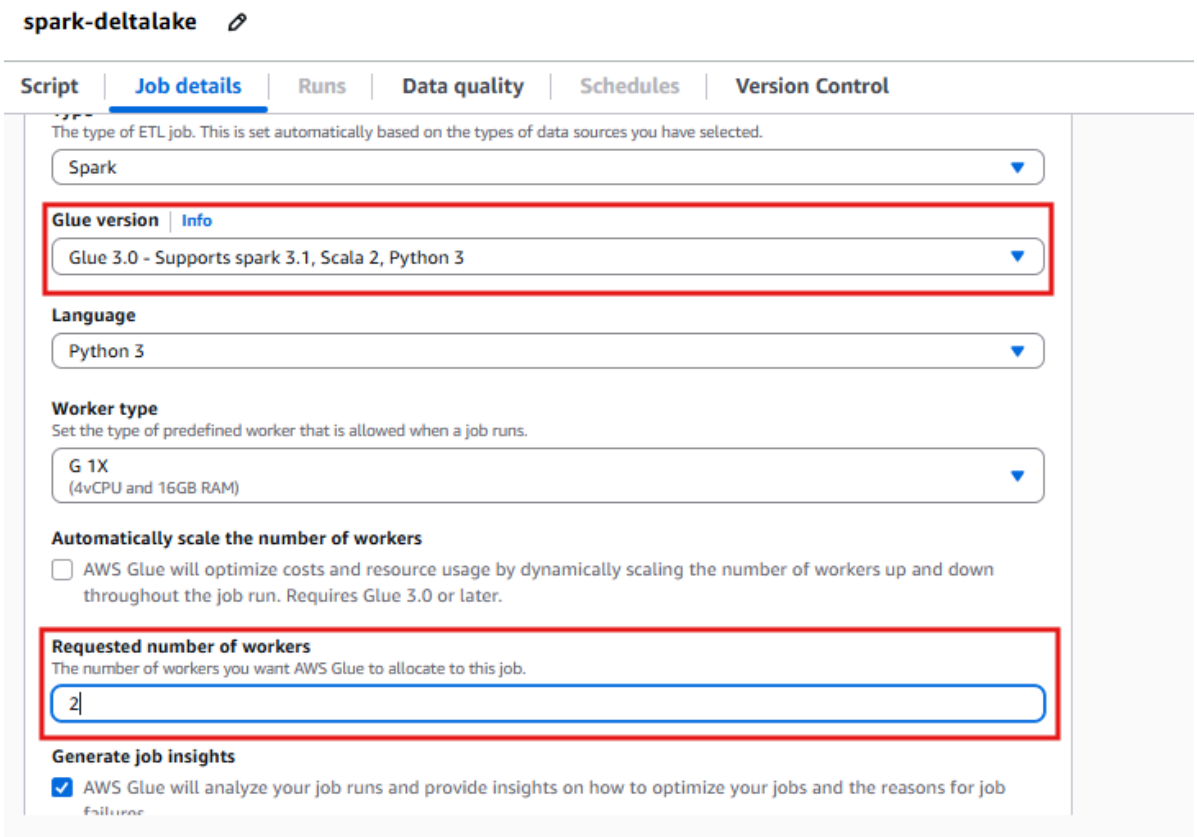
Make changes:

- Uncomment dynamic table name argument.
- Remove hardcoded table name.

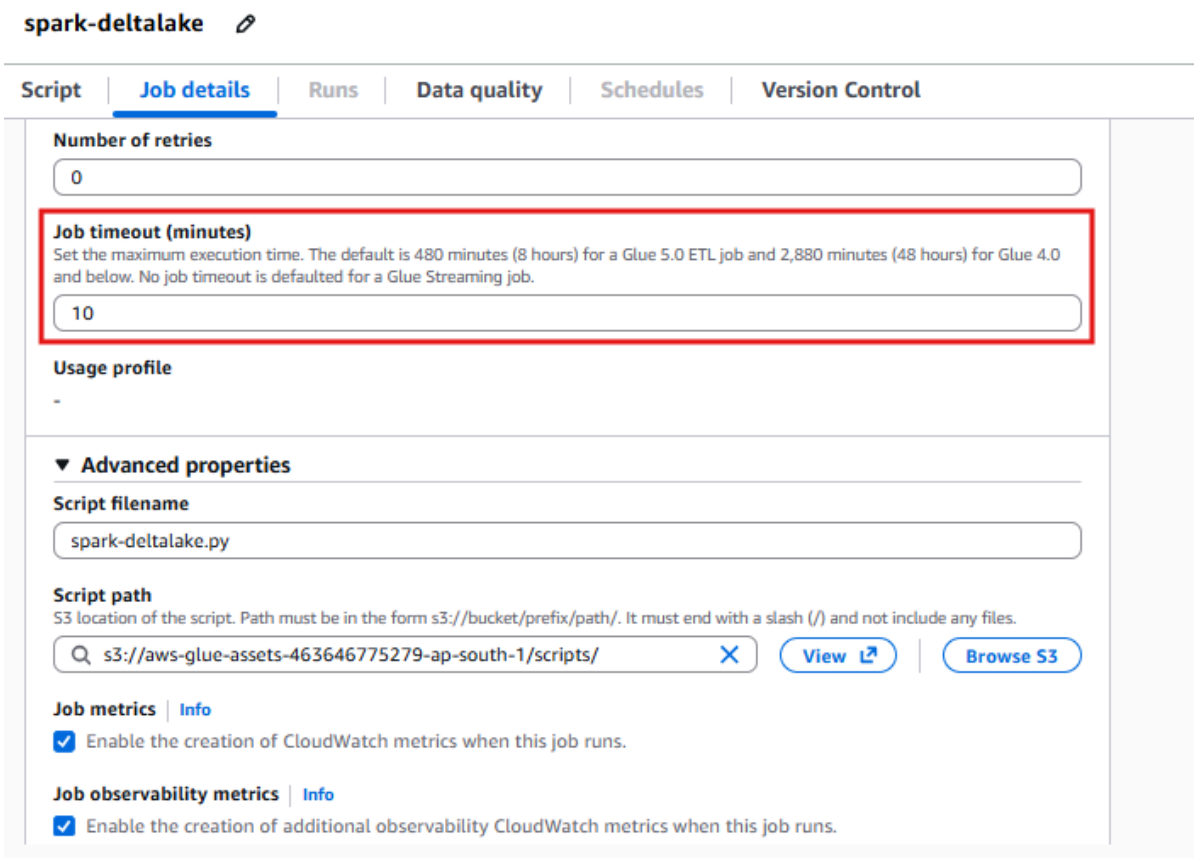
5. Go to job details tab —→ Name the job —→ Select the IAM role



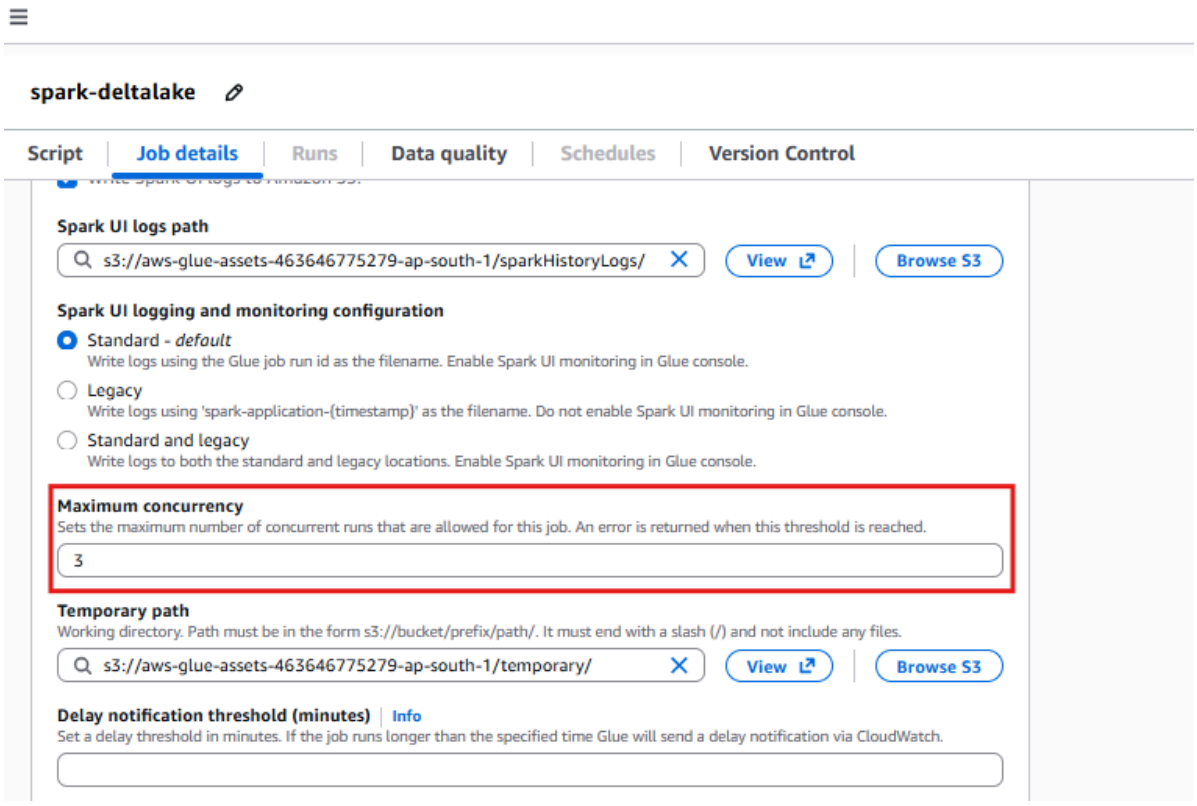
6. Glue version: 3.0 —→ Requested number of Workers: 2.



7. Timeout: 10 minutes

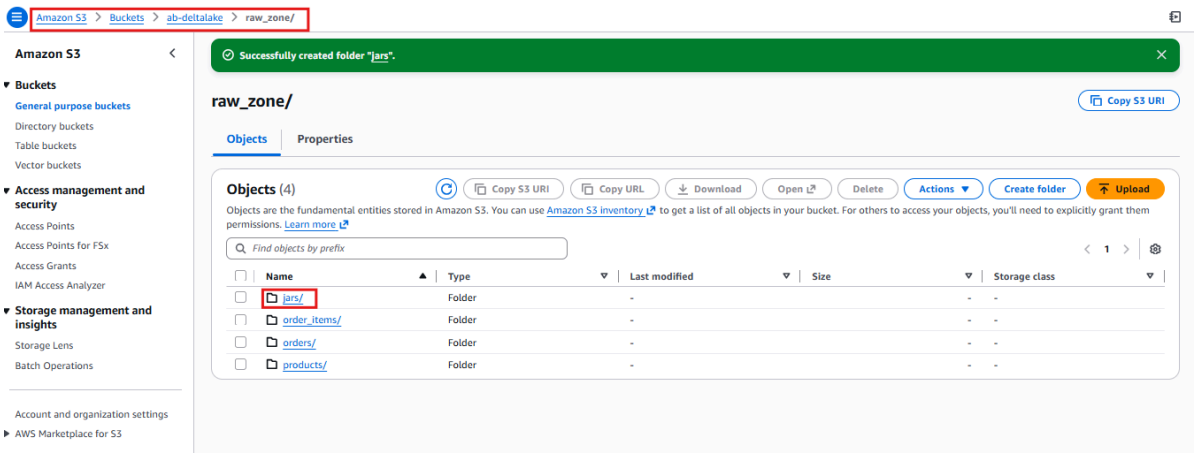


8. Max concurrency: 3.

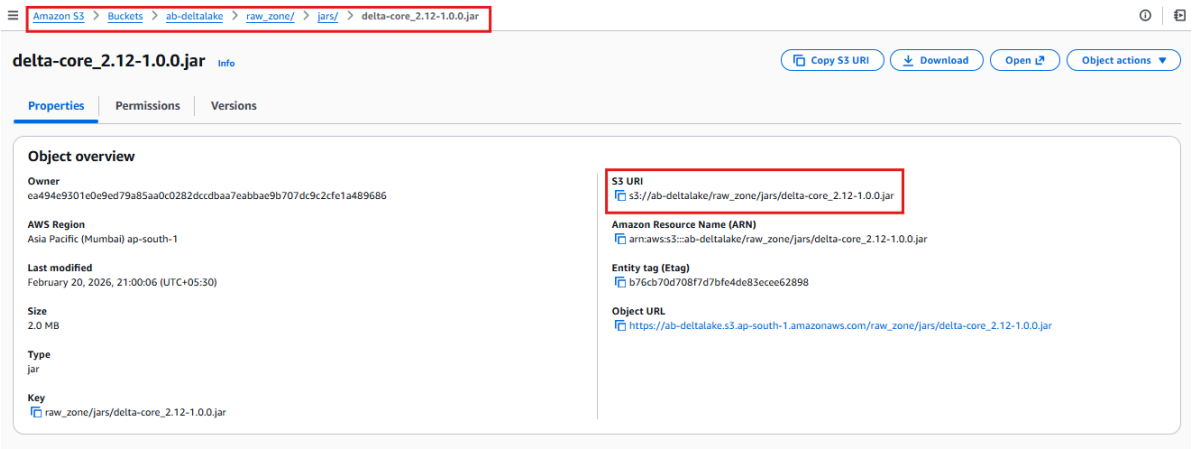


Step 10 — Add Delta Jar Dependency

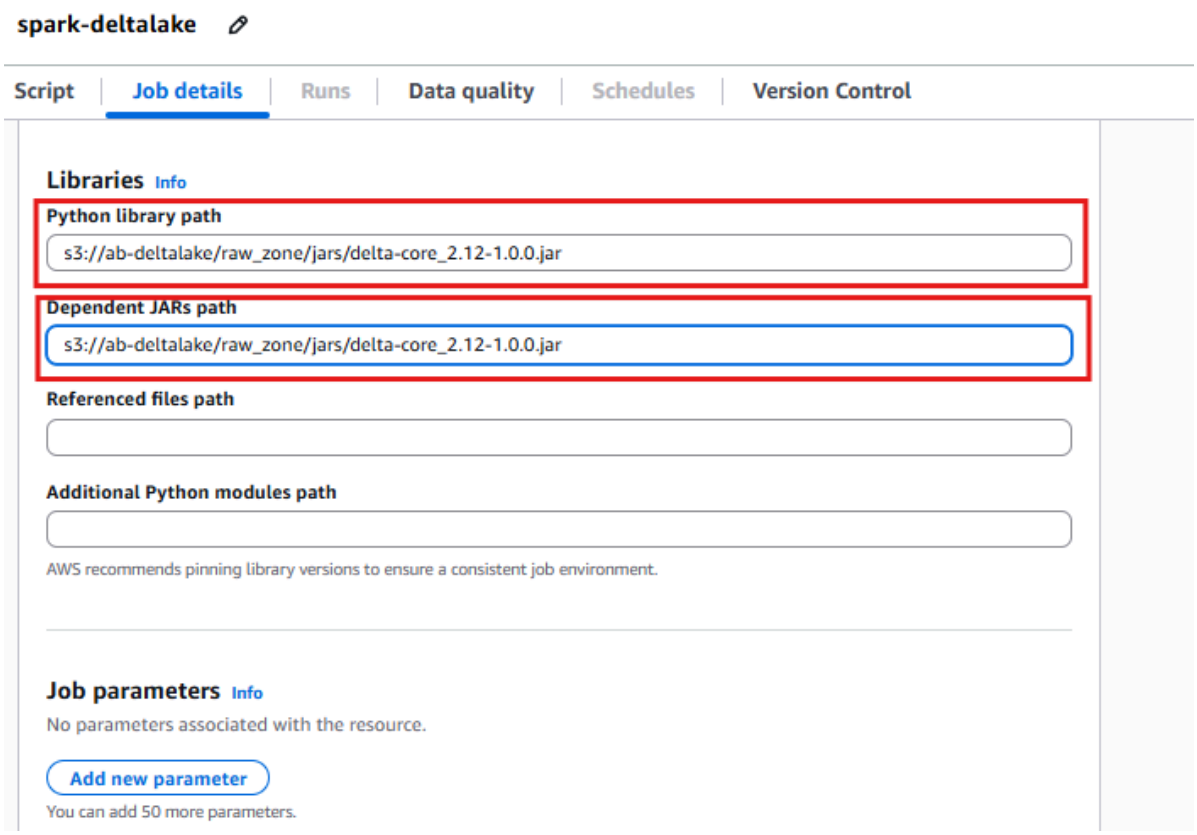
1. Upload jar file to S3 inside jars folder.



2. Copy S3 URI.

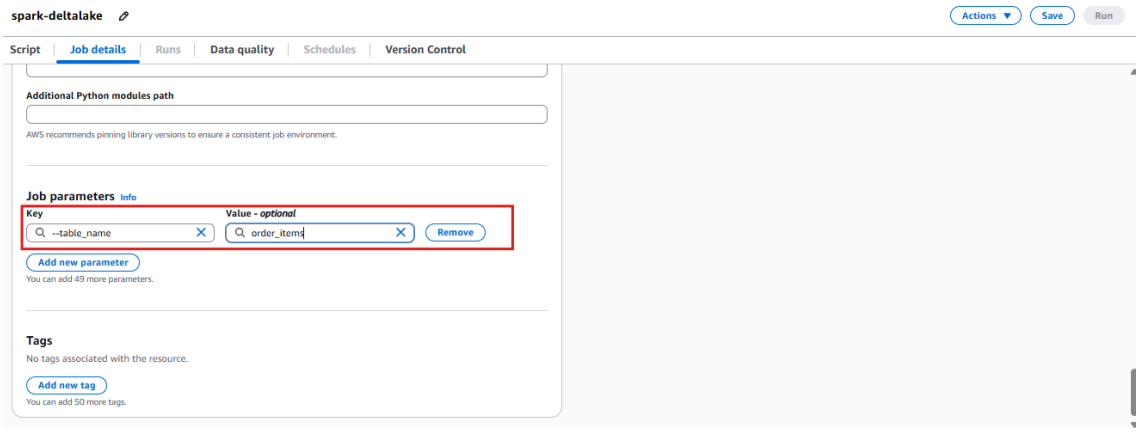


3. Add URI in job libraries section of Glue job

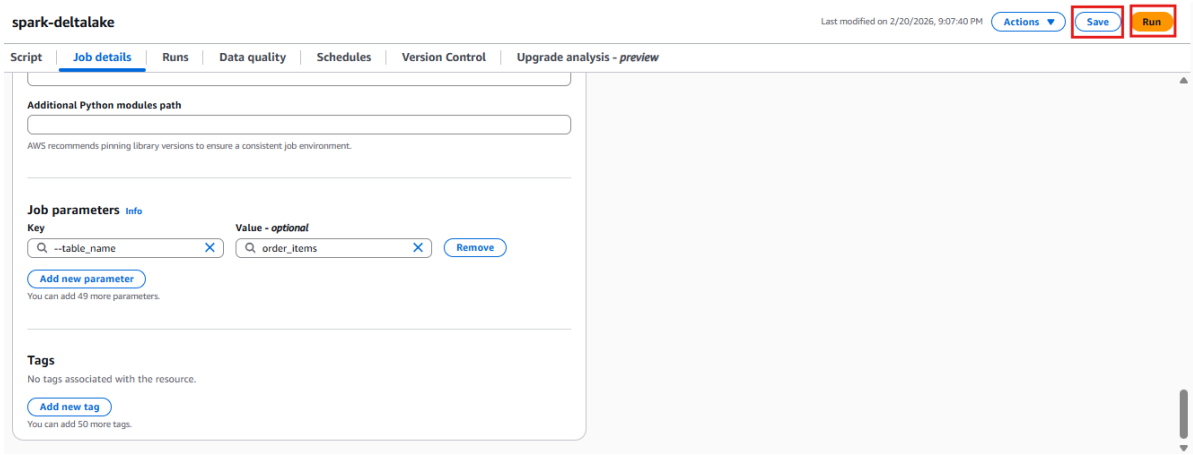


Step 11 — Add Job Parameters

1. Add parameter:
    - -table_name = order_items



2. Click save —→ Run job.

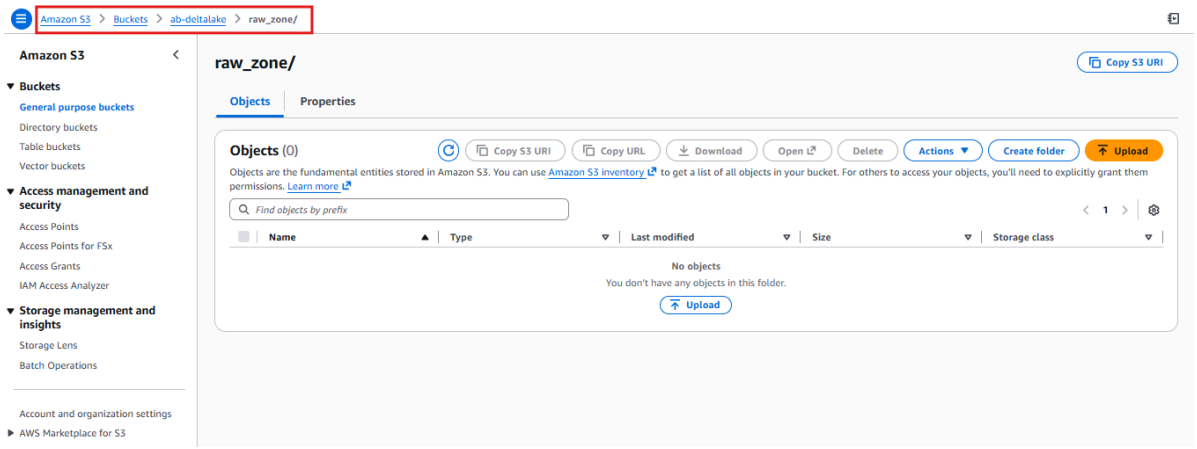


3. Repeat for:

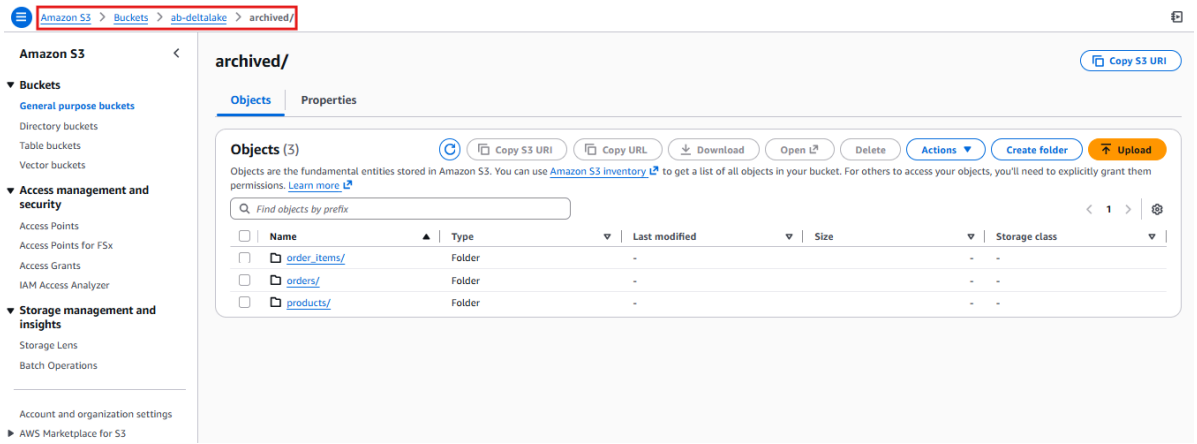
- products
- orders

Step 12 — Validate Data in S3

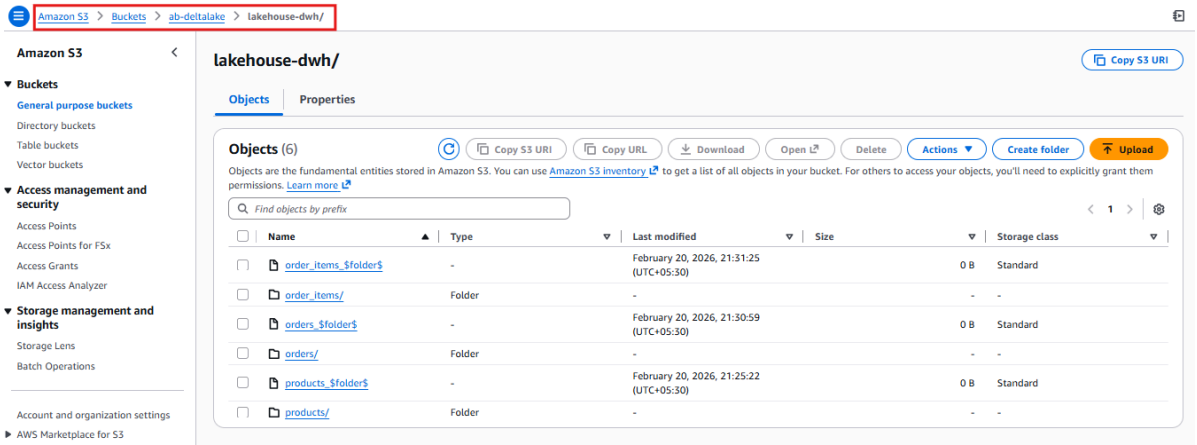
1. Open raw_zone → should be empty.



2. Open archive → should contain processed files.

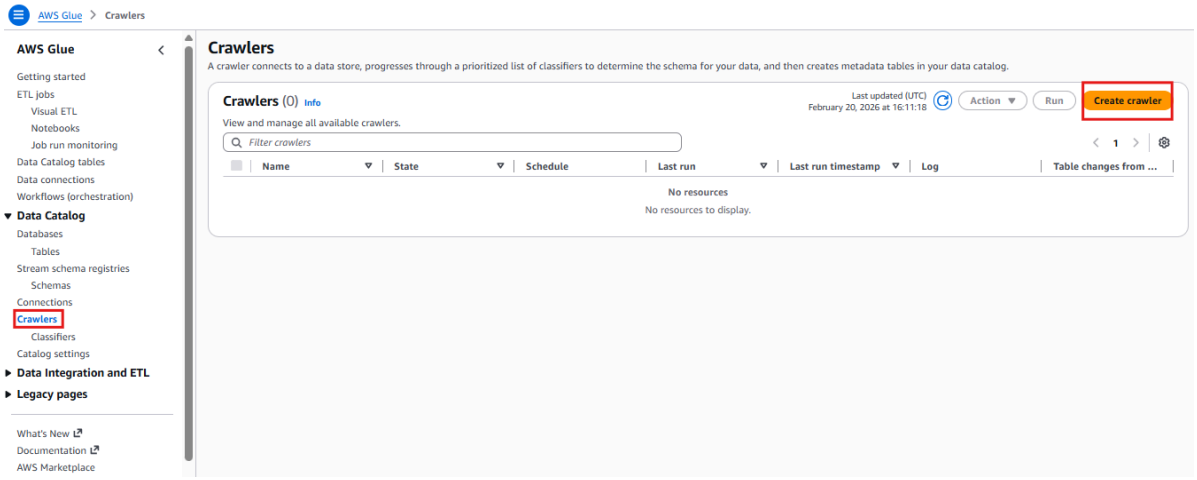


3. Open lakehouse_dw → should contain 3 tables.

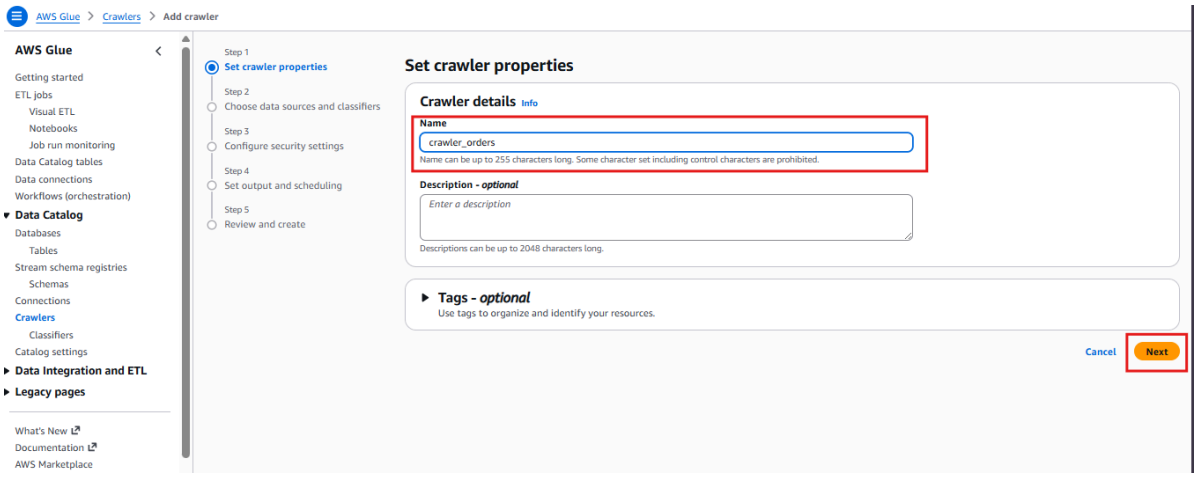


Step 13 — Create Glue Crawlers

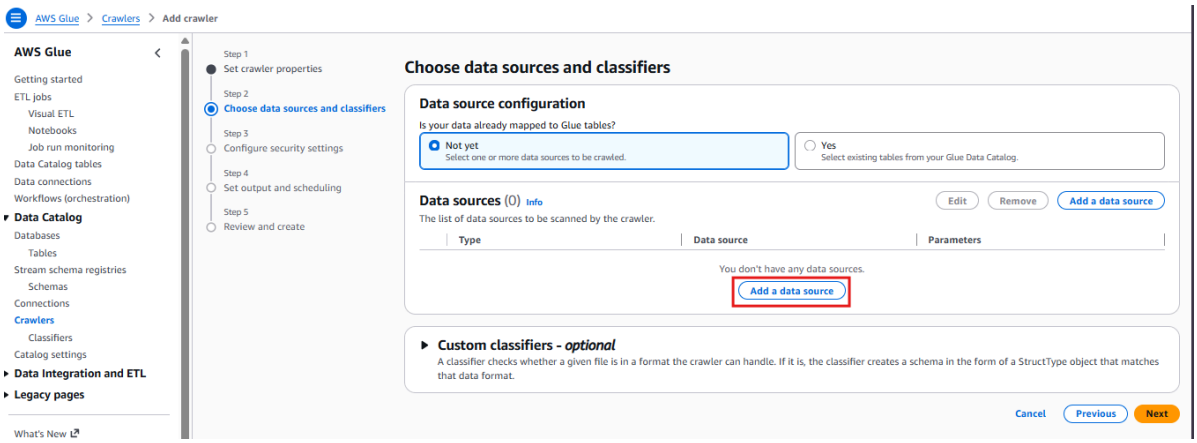
1. Go to Glue → Crawlers.



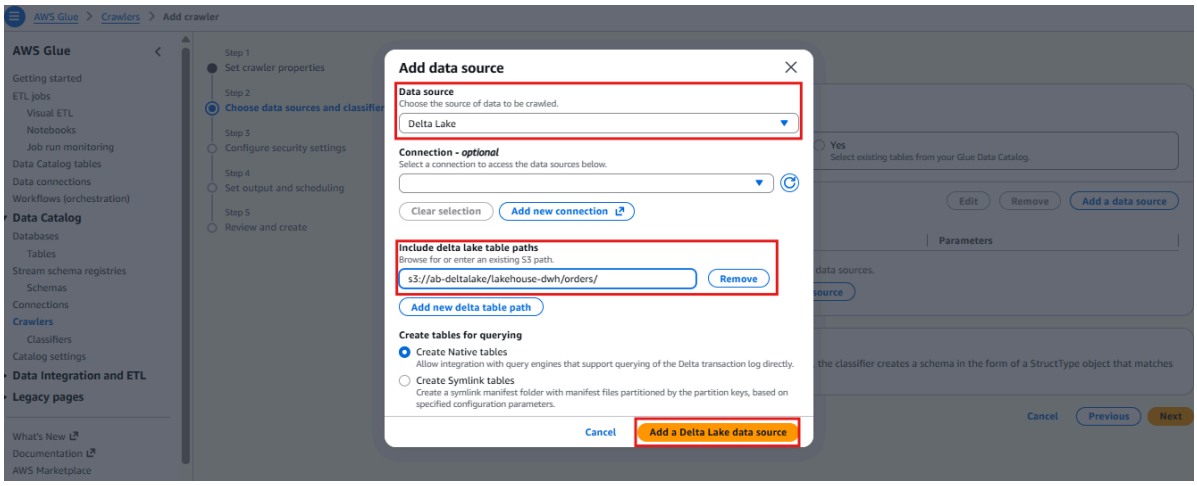
2. Create crawler for orders.



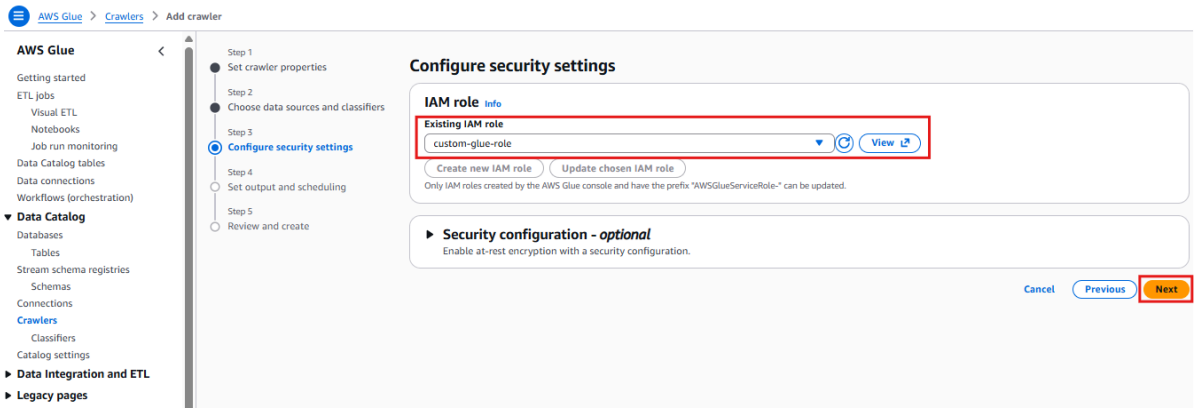
3. Data source



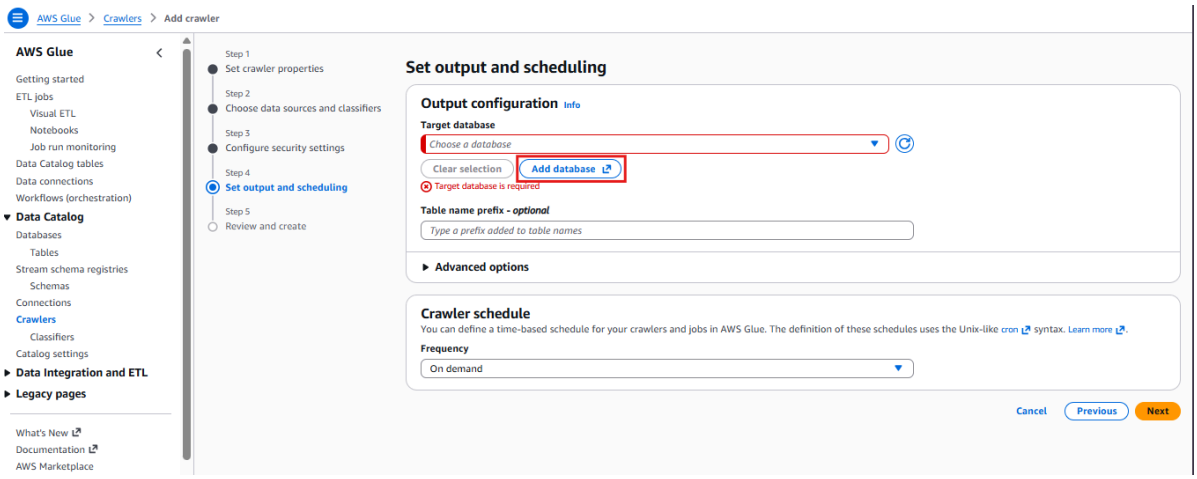
4. Select Delta Lake —→ Provide S3 path of **orders** table —→ Click next



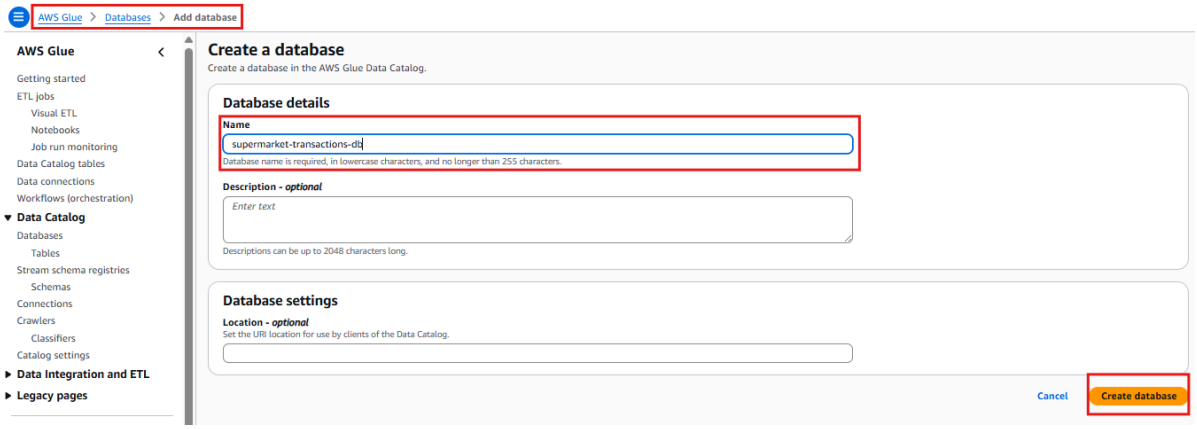
5. Select the IAM role —→ Click Next



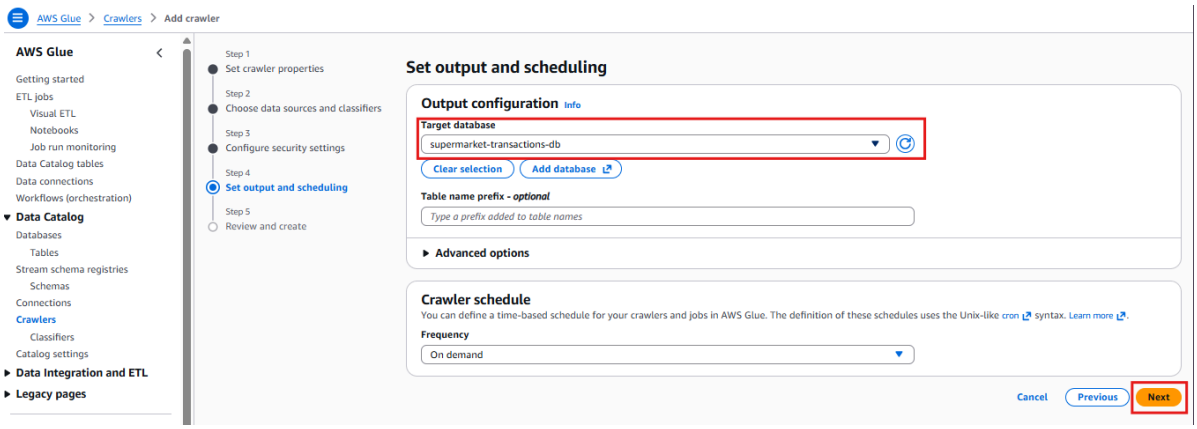
6. Select target database —→ Click **Add database**



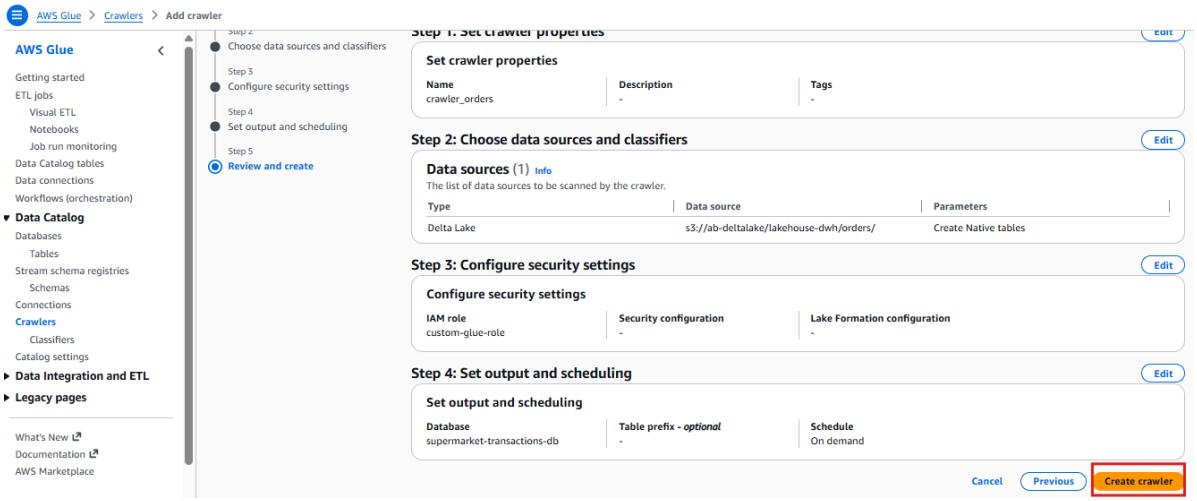
7. Name database: supermarket-transactions-db —→ Click **Create database**



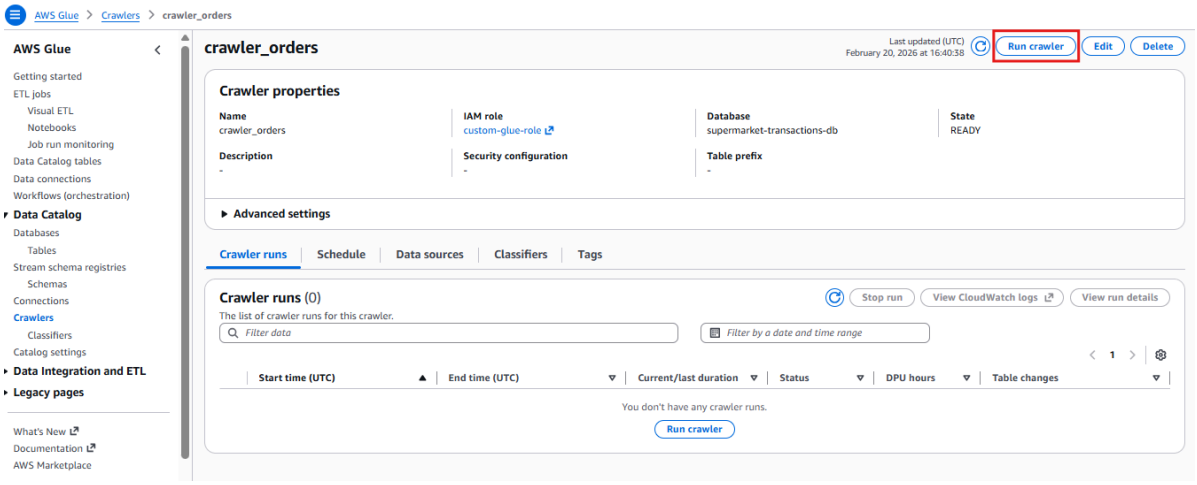
8. Once done creating database —→ Navigate back to crawler creation page —→ Select database



9. Click next —→ Create crawler



10. Run crawler.

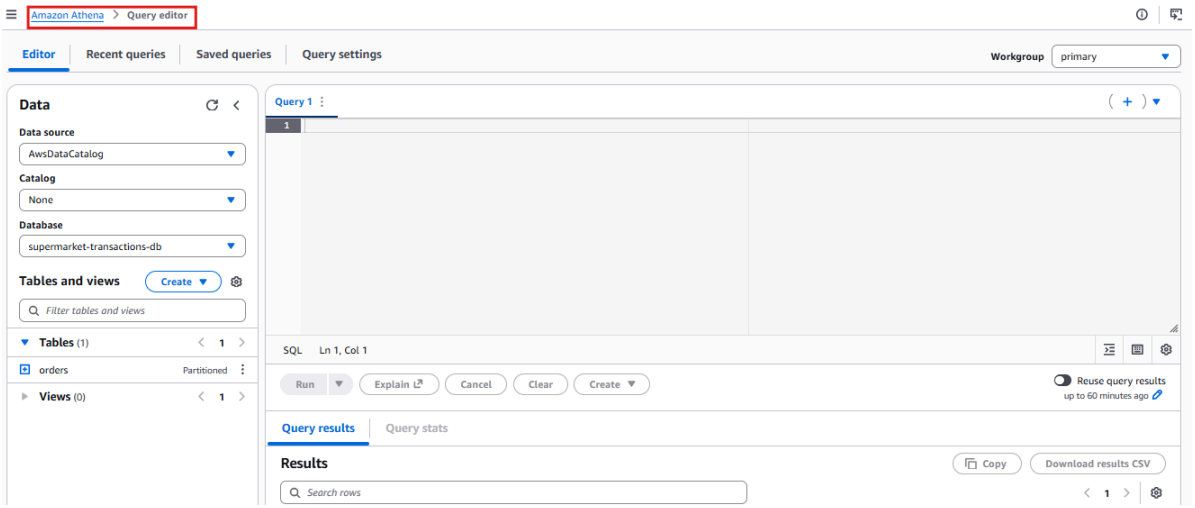


11. Repeat same steps for:

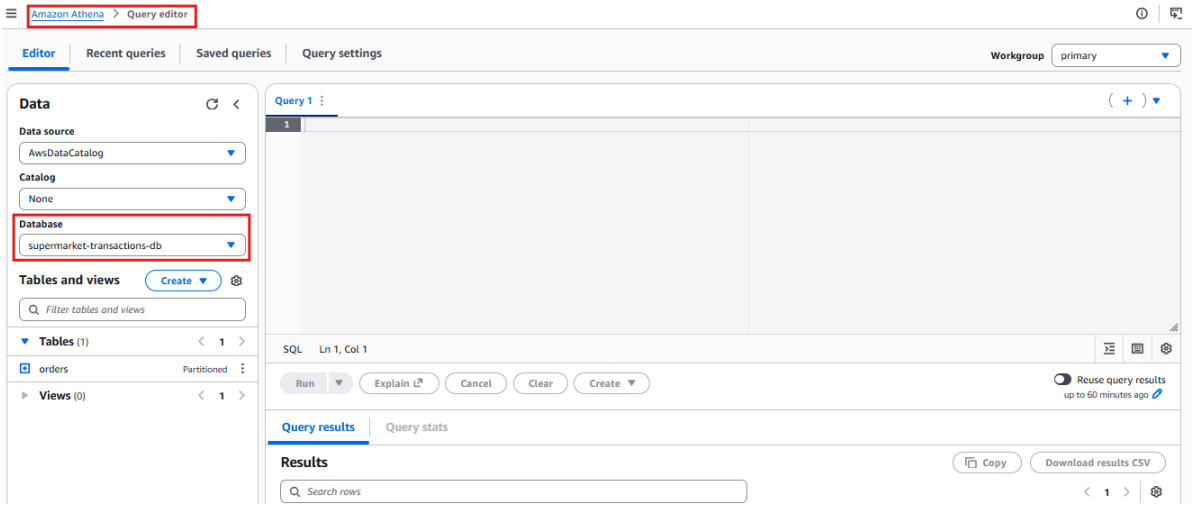
- order_items
- products

Step 14 — Query Data in Athena

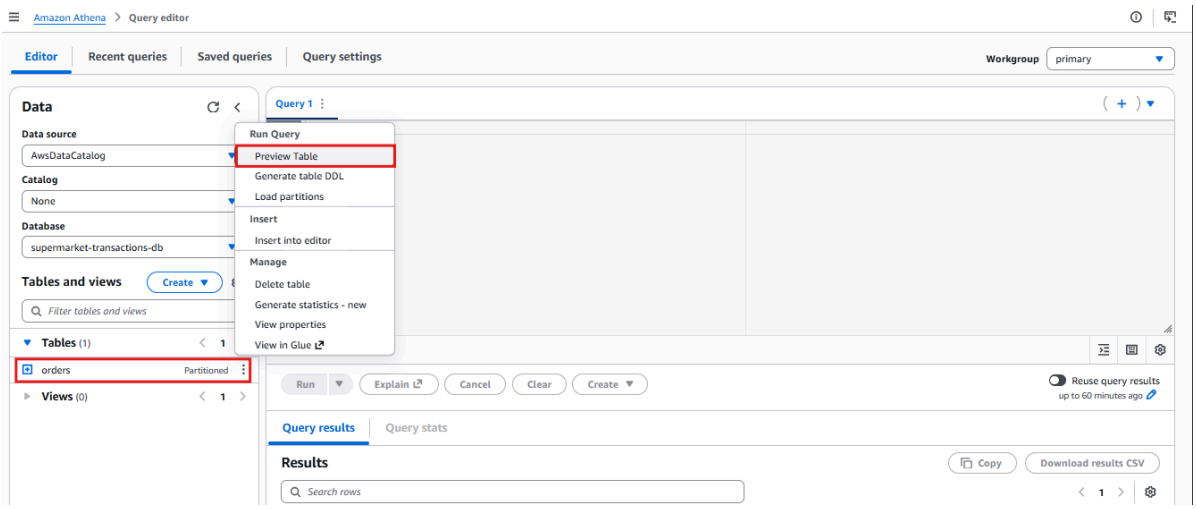
1. Open Athena —→ query editor



2. Select database.

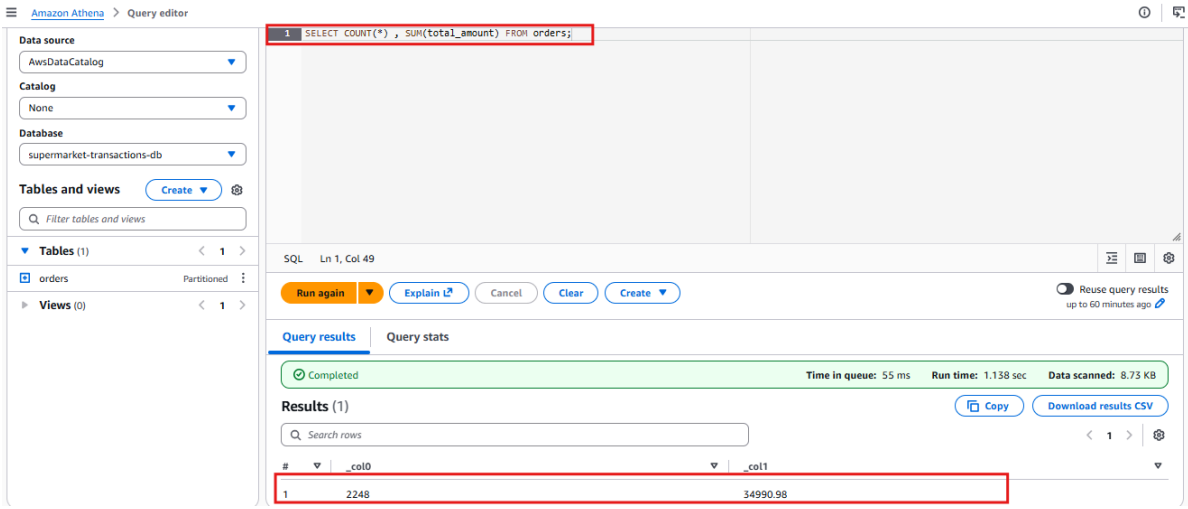


3. Preview orders table.



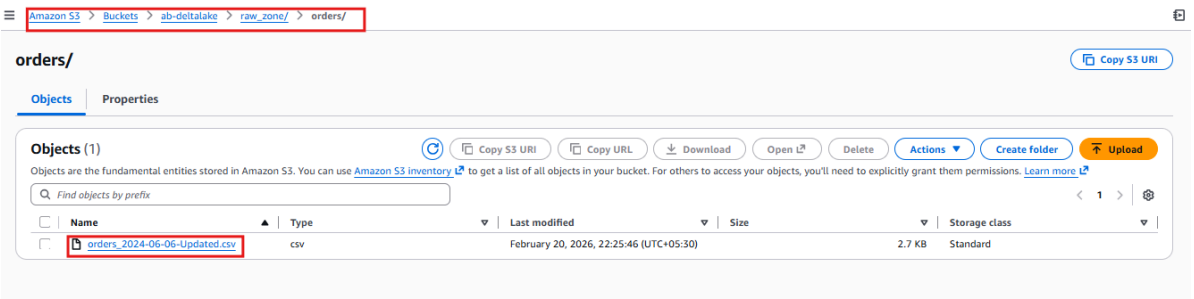
4. Run query:

   SELECT COUNT(*) , SUM(total_amount) FROM orders;

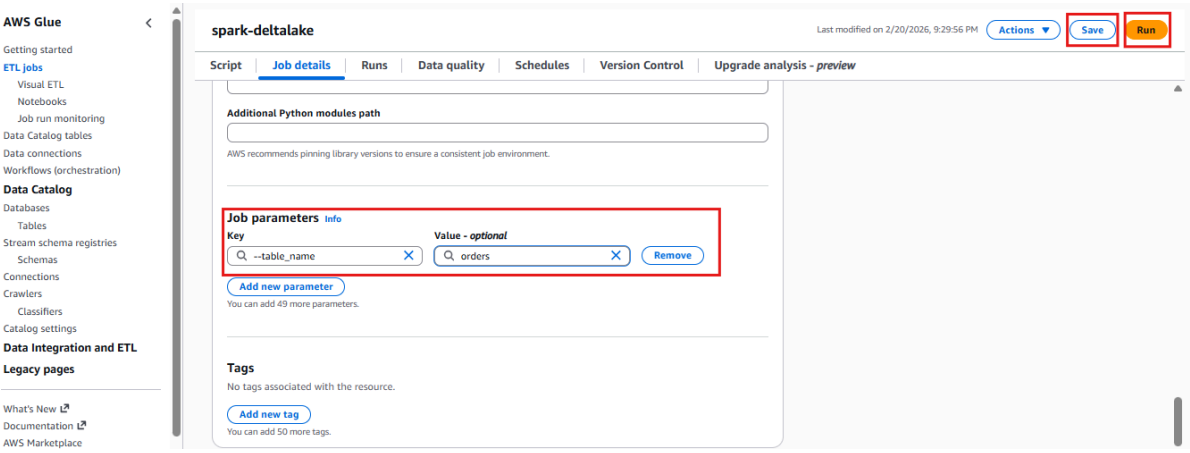


Step 15 — Test Delta Merge (Upsert)

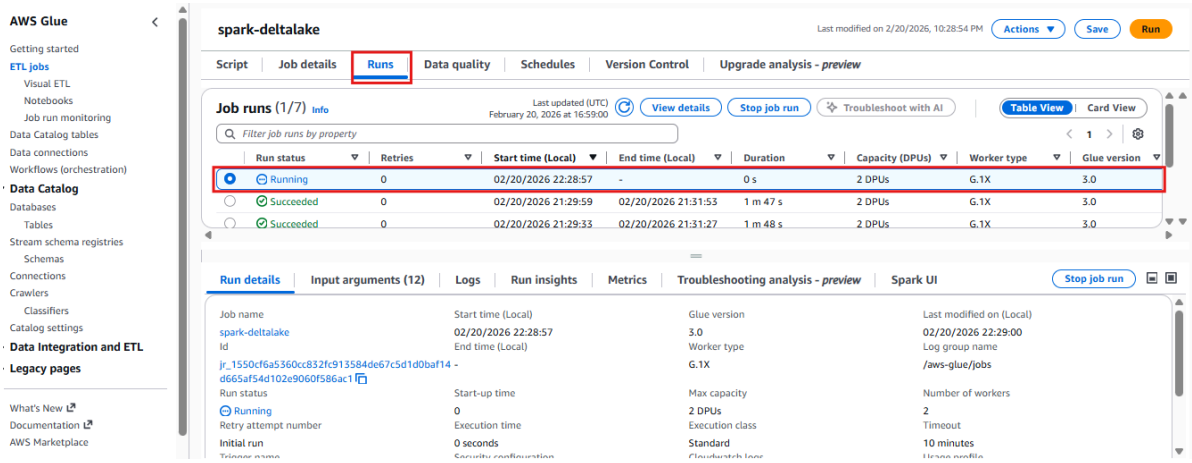
1. Open updated orders CSV.
2. Upload file to raw_zone/orders.



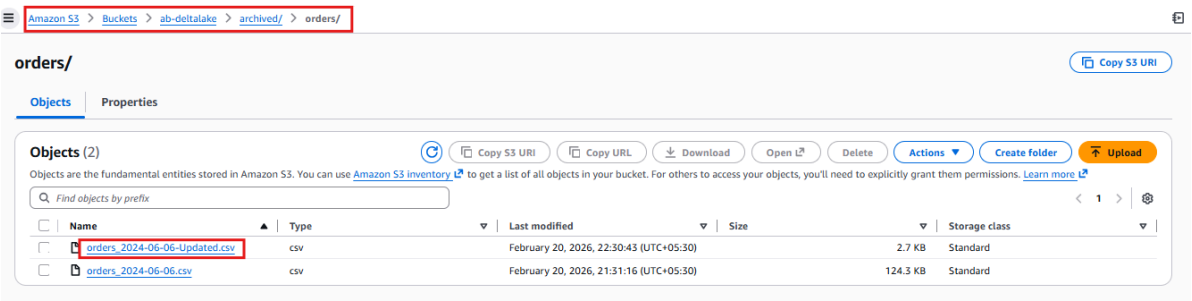
3. Run Glue job with table_name = orders.



4. Wait for job completion.

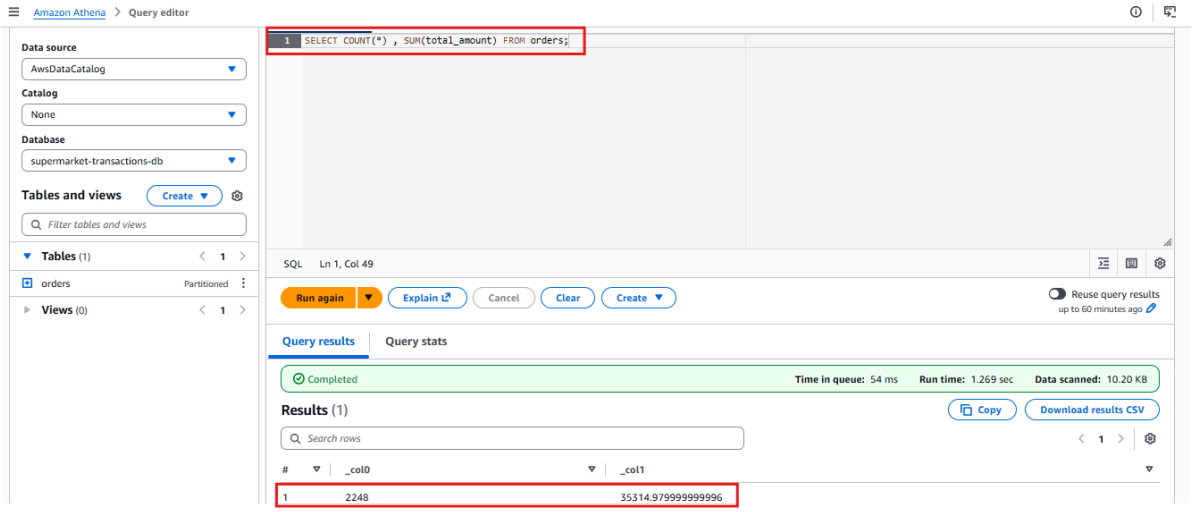


5. Verify file moved to archive.



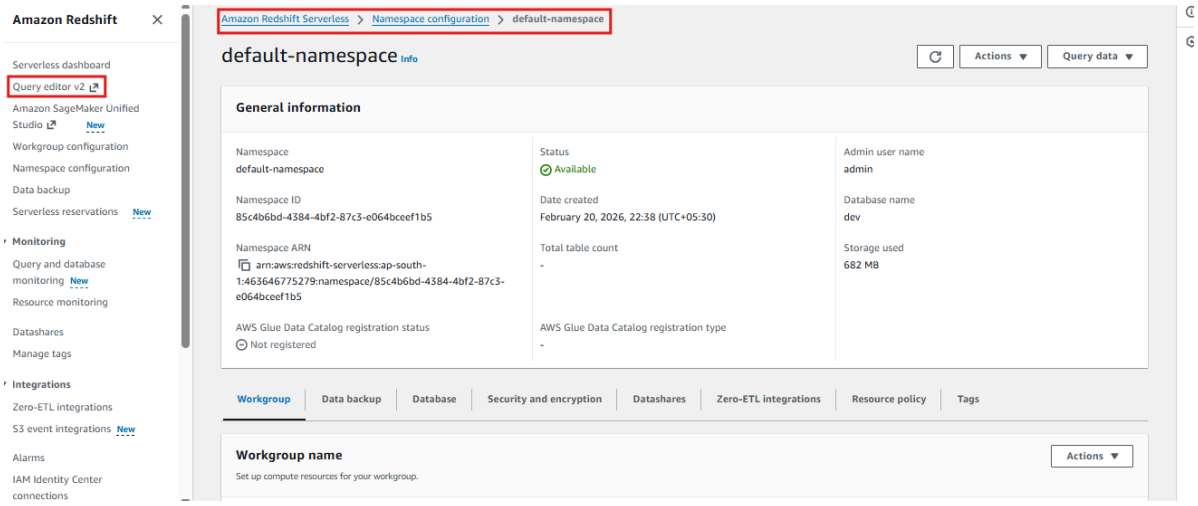
Step 16 — Validate Updated Data

1. Run same query in Athena.
2. Confirm:
   - Order count same
   - Revenue increased

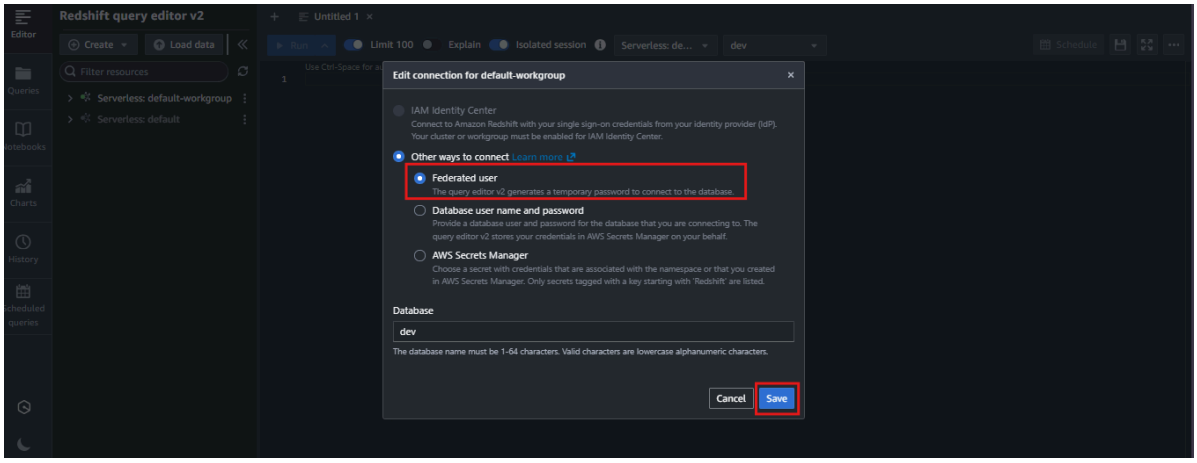


Step 17 — Access Data from Redshift

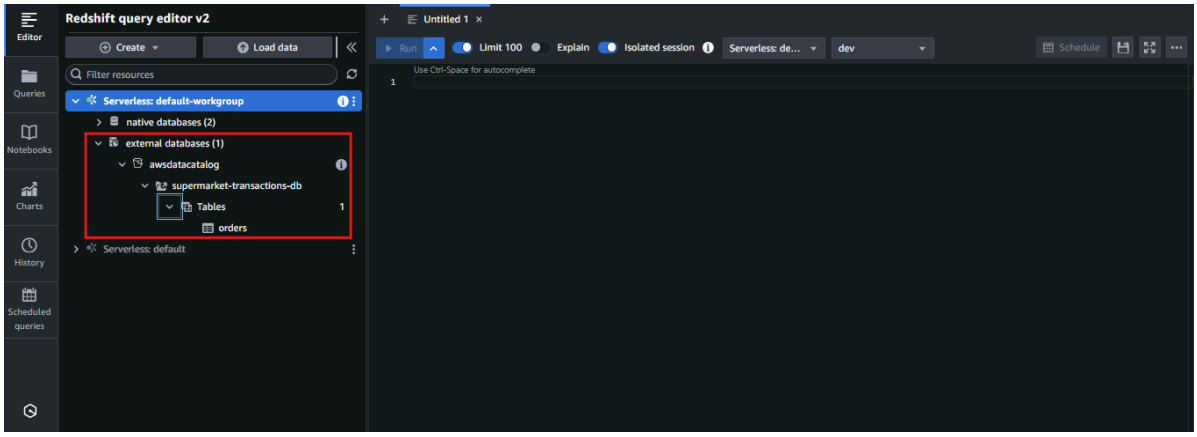
1. Open Redshift Query Editor.



2. Edit connection → Federated user.



3. Expand AWS Data Catalog —→ View database tables.



Step 18 — Workaround for Querying Delta Tables

1. Open Athena.
2. Run:

   ```
   CREATE TABLE delta_lake_orders AS
   SELECT * FROM orders;
   ```

3. Refresh Redshift schema.
4. Query new table successfully.

Output

After completing this lab you will have:

✅ Lakehouse architecture implemented

✅ Delta tables stored in S3

✅ Glue Data Catalog tables created

✅ Successful Athena queries

✅ Demonstrated ACID merge capability

✅ Archived raw files

✅ Redshift accessible table workaround